

DRAS: Deductive Reasoning Agent System

Experiment Report

1. Motivation and Goal

Language models are trained inductively: they learn to pattern-match over a training distribution and generate text that is statistically likely given a context. This makes them unreliable for multi-step deductive reasoning — they will frequently skip steps, confuse premises, or hallucinate intermediate conclusions that sound plausible but are not entailed by the given facts.

The specific value we are trying to produce is **reliable, auditable deductive reasoning traces that naturally accommodate natural language complexity**. "Reliable" means each step is logically valid and can be independently verified. "Auditable" means each step in the proof is attributed to its supporting premises, making the full derivation inspectable by a domain expert. "Natural language complexity" means the system operates on ordinary declarative sentences — not formalized predicates, Prolog clauses, or structured schemas — so it can be applied to real-world professional, legal, scientific, and policy reasoning without a formalization step.

The central hypothesis is that separating *what to infer from* (retrieval) from *what follows* (inference) and validating each derived step before it enters the knowledge base gives deterministic, auditable reasoning without sacrificing the flexibility of natural language input.

2. Related Work

2a. Formal Logic Engines

Systems like **Prolog**, **Vampire**, and **Z3** perform sound, complete deductive reasoning but require premises expressed in formal syntax (predicate logic, Horn clauses, SMT theories). Translating natural language into these representations is itself an unsolved problem and the translation step is where errors are introduced. These systems produce no natural-language-readable proof trace.

LangPro (Abzianidze, 2017) applies the analytic tableau method to natural logic and can parse natural language directly, but is restricted to a narrow logical fragment and struggles with the vocabulary variation common in real-world reasoning ("failed the inspection" and "did not pass the inspection" must be explicitly equated).

Answer Set Programming (Clingo) offers more expressivity but the same formalization bottleneck. All formal systems are brittle to the linguistic variation inherent in real-world knowledge.

2b. Neural and LLM-Based Reasoning

Chain-of-Thought prompting (Wei et al., 2022) elicits step-by-step reasoning from large models but does not validate intermediate steps — there is no mechanism to prevent a hallucinated conclusion from propagating. Recent work shows CoT gains diminish as model scale increases and that shorter, more careful

chains often outperform longer ones, suggesting that generation quality rather than chain length is the binding constraint.

Selection-Inference (Creswell et al., 2022) separates which premises to use from what to conclude, running two models in alternation. Architecturally related to DRAS but remains single-pass: it does not maintain an iterative growing knowledge base, and steps are not independently validated.

LAMBADA (Kazemi et al., 2023) uses backward chaining with few-shot LLMs: starting from the goal, it decomposes reasoning into four modules (Fact-check, Prove, Extract, Aggregate) and works backward to axioms. It produces step-level traces and is more efficient than forward chaining on long proofs. However, it relies on the LLM's implicit reasoning quality at each step without fine-tuning for atomic inference, and does not maintain a growing validated store.

LINC (Olausson et al., 2023) is the most capable neurosymbolic hybrid for natural language: an LLM acts as a semantic parser to translate premises into first-order logic, then hands off to an external theorem prover (Prover9). This produces a formally verified proof but the chain is only as good as the NL→FOL translation — any parser error breaks the entire chain, and the approach requires a formal target language that constrains which reasoning patterns are expressible.

ProofWriter (Clark et al., 2020) fine-tunes a T5 model to generate multi-step proofs from structured premise sets. It produces auditable traces but is trained to generate the entire chain in one pass, which limits generalization to novel premise configurations, and requires rigidly formatted input.

2c. Neurosymbolic Hybrids

Logical Neural Networks (LNN, IBM) embed logical operators as differentiable neurons. Produce interpretable traces tied to logical structure but require upfront formalization — there is no built-in NL-to-logic translation.

Neural Theorem Provers (NTP) and **DeepProbLog** combine learned embeddings with probabilistic logic programming. Powerful for structured domains but do not generalize to open-domain NL premises.

2d. The Gap

Property	Prolog/Z3	CoT	LAMBADA	LINC	ProofWriter	DRAS
Natural language input	X	✓	✓	✓ (via parsing)	Partial	✓
Validated atomic steps	✓	X	X	✓ (formal)	X	✓
Iterative growing store	X	X	X	X	X	✓
Attributed proof trace	✓	X	Partial	✓	✓	✓
Handles NL variation	X	✓	✓	X	Partial	✓

DRAS is designed to fill this gap with a fine-tuned small LLM as the inference engine and a retrieval-augmented loop as the reasoning controller.

3. System Architecture

3a. Core Design Principle

DRAS decomposes multi-step deductive reasoning into a loop of three operations:

1. **Retrieve** — given the current goal and last derived fact, pull the most relevant premises from an in-memory knowledge store
2. **Infer** — ask the fine-tuned model: "given these premises and this goal, what is the next valid inference?"
3. **Validate and Store** — before committing the result, independently verify it; if valid, add it to the store and repeat

This decomposition produces two properties:

- **Local validity**: every stored step is independently checked against its generating premises — hallucinations cannot silently compound
- **Compositionality**: the retriever's job is to find useful premises at each step; the model's job is only to make one valid inference from what it is shown; the controller's job is to route between them

The key insight behind the design is that a model trained to make one reliable atomic step is more tractable than a model trained to generate full multi-step proofs. A verified atomic step is a permanent gain; an unverified multi-step generation may be wrong at step 3 of 6 and there is no mechanism to catch it.

3b. The Retriever

A dense retriever ([BAAI/bge-small-en-v1.5](#)) embeds all premises in the knowledge store and retrieves the top-k most similar to a weighted combination of the **goal** and the **last derived fact**. The goal-bias (`k_goal=1`) ensures the retrieval is always anchored toward the final target while the step-proximity bias keeps the current reasoning thread relevant.

This dual-anchor retrieval (`query_multi`) is necessary because the goal alone underweights intermediate facts — early in a long chain, the first step is semantically distant from the goal and would never be retrieved. The `last_derived` anchor surfaces the bridge rules that extend the chain from its current position.

3c. The Inference Model

A 7B parameter DeepSeek-R1-Distill-Qwen model, fine-tuned with LoRA ($r=16$, all attention and MLP projections) on a structured prompt:

```
<|goal|>{conclusion}
<|premises|>{premise_1} {premise_2} ...
<|next_step|>{next valid inference}<|end|>
```

Loss is masked to the `<|next_step|>` completion only, so the model learns a single function: given a set of premises and a goal, generate the one valid inference that most directly follows and advances toward the goal. It is not asked to generate a full proof. It is explicitly trained to refuse when no valid inference follows from the given premises.

3d. The Validator

Before any generated step enters the knowledge store, it passes three checks:

1. **Well-formedness**: rejects empty strings, questions, hypotheticals, and the refusal string itself
2. **Grounding**: verifies that no named entity or key noun phrase appears in the generated step that was not present in the supporting premises — this is the primary guard against hallucinated concepts bleeding into the store
3. **LLM validity**: asks the model in a discriminative prompt ("does this inference follow from these premises?") — the same model that struggles to *generate* a specific conclusion in some configurations can reliably *validate* it, exploiting a generation/validation asymmetry

A step that fails any check is discarded; the loop retries with sampling or triggers a recovery mechanism.

3e. Recovery Mechanisms

The main loop will stall when the model repeatedly fails to generate a valid step — typically because the goal is too far from the current store to bridge in one step, or because the required next step requires a non-obvious inference pattern. Four escalating recovery mechanisms handle this:

Sampling fallback (`consecutive_rejects` ≥ 3): switches from greedy decoding to temperature sampling ($T=0.7$). Greedy decoding is a mode-seeking decoder; when it is wrong it is consistently wrong. Sampling explores the nearby probability mass and often surfaces a valid step that greedy missed. Analogous to a human pausing to reconsider rather than following the same chain of thought that already failed.

Backward chaining (`consecutive_rejects` = 6, 12, 18): identifies the conditional rule most similar to the current anchor (the last derived fact), then runs a focused 2-premise inference with only `[anchor_fact, bridge_rule]` — no distractors. This mirrors the mathematical strategy of targeting an intermediate lemma: instead of asking "what gets me to the goal from here?", ask "what does this specific rule produce when applied to what I currently know?" A conditional filter discards any output that looks like a rule rather than a specific derived fact — a hallucination pattern where the model generates rule compositions instead of applying a rule to an entity.

Goal projection (`consecutive_rejects` = 9, 18, 27): attacks the problem from the other direction. It identifies rules proximate to the final goal (rules whose consequents are semantically close to what we need to prove) and either (a) asks the model to generate the antecedent of each such rule as an intermediate sub-goal — reorienting the retrieval anchor to a closer waypoint — or (b) falls back to a deterministic modus ponens application: parse the rule's antecedent and consequent via regex, check whether the antecedent matches the current derived fact by cosine similarity (threshold ≥ 0.68), and if so, construct the conclusion directly as "`[entity] [consequent]`." without model involvement. A forward-chaining while loop then re-applies immediately after each success, chaining multiple steps in one trigger. This is analogous to a planner working backward from the goal to identify the next concrete milestone, then executing forward from it.

Exhaustive pairwise search (`consecutive_rejects` = 12, 24, 36): last resort. Tries every ordered pair of premises in the current store via focused 2-premise inference. BFS-expands valid new steps — each newly found fact is immediately paired with all existing premises — so compound inferences are discoverable within a single trigger. A 10-step cap prevents BFS explosion on unprovable problems. Analogy: when all targeted search has failed, enumerate the space exhaustively.

3f. Why These Are Sufficient

Together, the mechanisms constitute a complete search strategy over the logical space reachable from the current store:

- **Sampling** covers generation variance within the model's distribution
- **Backward chaining** covers focused application of a specific applicable rule — a systematic one-hop search anchored at the current position
- **Goal projection** covers the case where the next applicable rule is not near the current anchor but is near the goal — a heuristic long-range jump
- **Deterministic rule instantiation** covers the model's known failure mode (modus ponens in generation mode) with a sound procedural fallback
- **Pairwise search** is provably complete over one-step derivations from the current store

No single mechanism handles all cases; together they cover the space. The ordering by consecutive_reject threshold means cheap mechanisms fire first (sampling is essentially free; backward chaining is $O(N)$ inference calls; pairwise is $O(N^2)$) and expensive mechanisms fire only when cheap ones have been exhausted.

The architecture is also conservative by design: false proofs are bounded by the validator (a step must be independently verified before entering the store), and the unprovable termination condition is MAX_ITER (the loop simply runs out of iterations when no valid chain exists).

4. Training

4a. Data Sources

FOLIO ([tasksource/folio](#) on HuggingFace): ~1,430 natural language reasoning problems with labels True / False / Uncertain. True-labeled examples contribute positive training instances (premises → conclusion, 1-step proof). Non-True examples contribute negatives where premises are borrowed from a *different* example so they provably cannot support the goal — the model learns a calibrated refusal output rather than confabulating.

Synthetic chains ([dras/synth.py](#), [dras/synth_extended.py](#)): hand-authored chains covering five inference patterns at 15-20 examples per pattern:

- Modus ponens chain (forward, 3-6 steps)
- Modus tollens / contrapositive instantiation
- Contrapositive chain (backward reasoning)
- Disjunctive syllogism (with explicit negation steps)
- Mixed patterns (combining the above)

Each chain generates one training example per step: `premises = seeds + all_prior_steps`, `target = next_step`. This teaches the model to reason from the accumulated store, not just from the original seeds.

Annotated chains (oracle masking): a subset of chains with explicit `support` fields per step. These generate oracle training examples where the model sees only the 2-3 premises that directly support that specific step, not the full accumulated context. Oracle examples are upweighted ($\times 2$) because they provide the tightest possible learning signal for atomic inference.

Final dataset: ~820 train / ~145 eval rows. The training objective is completion-only (`DataCollatorForCompletionOnlyLM` masking on `<|next_step|>`), so only the generated inference token

is in the loss — the model never trains to predict premise or goal text.

4b. Training Setup

- Base: `unsloth/DeepSeek-R1-Distill-Qwen-7B-bnb-4bit`
- LoRA: `r=16`, `alpha=16`, zero dropout, all attention + MLP projection layers (`q/k/v/o_proj`, `gate/up/down_proj`)
- Trainer: TRL SFTTrainer, batch size 1, gradient accumulation 16 (effective batch = 16), `lr 2e-4`, 3 epochs, BF16, no packing
- Hardware: RTX 3090 Ti

4c. Evaluation During Training

The reasoner is evaluated on a held-out 15% split with oracle premises (the model is given exactly the premises that support the ground-truth next step). This measures atomic step accuracy independently of retrieval quality, isolating the model's inference capability from retrieval errors.

5. Benchmark Evaluation

Nine benchmark problems test the full end-to-end system (retriever + reasoner + loop controller). They are organized by logical pattern and difficulty tier.

ID	Pattern	Description	Steps	Provable
B1	<code>modus_tollens</code>	Contrapositive instantiation from a universal rule	2	Yes
B2	<code>hypothetical_syllogism</code>	3-step forward chain through conditional rules	3	Yes
B3	<code>disjunctive_syllogism</code>	Eliminate one disjunct, then 2-step forward chain	3	Yes
B4	<code>contrapositive_chain</code>	4-step backward derivation through negated consequents	6	Yes
B5	<code>mixed_negation</code>	Contrapositive then forward consequence chain	5	Yes
B6	<code>long_forward_chain</code>	6-step forward chain (Verdania climate treaty)	6	Yes
B7	<code>unprovable</code>	No logical connection between premises and goal	0	No
B8	<code>multi_pattern_ds_forward</code>	DS elimination + 3-step forward chain	5	Yes
B9	<code>multi_pattern_cp_forward</code>	2-hop contrapositive + 2-step forward chain	6	Yes

Each benchmark is run with `max_iterations=20` and the full recovery mechanism stack enabled (`--pair-search 12 --goal-proj 9`).

6. Results and Analysis

Final scores: `proof_completion_rate` = 1.0, `false_proof_rate` = 0.0

All 8 provable benchmarks reach `PROOF_FOUND`; the 1 unprovable benchmark correctly stalls at `MAX_ITER`.

ID	Status	Iterations	Mechanism responsible
B1	PROOF_FOUND	3	Main loop (backward chaining on iter 1, then direct step)
B2	PROOF_FOUND	2	Main loop (model-driven forward chain, all steps clean)
B3	PROOF_FOUND	19	Main loop + backward chaining (DS negation required multiple backchain triggers)
B4	PROOF_FOUND	10	Backward chaining (contrapositive rules require focused 2-premise context)
B5	PROOF_FOUND	2	Main loop (sampled after initial reject on the contrapositive step)
B6	PROOF_FOUND	12	Goal projection → deterministic rule instantiation, 3 chained steps
B7	MAX_ITER	20	Correct — no valid chain exists, pairwise search confirms
B8	PROOF_FOUND	10	Backward chaining for DS elimination, then model-driven forward chain
B9	PROOF_FOUND	2	Model-driven (contrapositive pattern well-represented in training data)

Pattern Analysis

What the model does well: The model handles modus ponens, hypothetical syllogism, and contrapositive instantiation reliably when the relevant premise is in the retrieved context. These patterns are well-represented in training data and structurally unambiguous. B2 and B9 are solved in 2 iterations — the model finds the correct next step immediately. B5 is solved in 2 iterations despite requiring a contrapositive, because the sampling fallback surfaces the correct step after one greedy rejection.

Where recovery mechanisms are essential: B3, B4, and B8 all involve disjunctive syllogism — the model must first negate one disjunct explicitly ("the hardware fault did not occur") before concluding the other. This two-step elimination is underrepresented in the model's greedy mode but is learned under focused 2-premise context. Backward chaining, which reduces the inference problem to `[last_derived, bridge_rule]`, reliably surfaces the elimination step that the full-context generation misses.

B6 is the stress case: 6 consecutive modus ponens applications where the model's generation mode fails at step 4 with a conditional output ("If a nation gains access to loans...") instead of the entity-specific fact ("Verdania can fund renewable energy projects."). This is a generation/validation asymmetry — the model trained for forward inference cannot reverse-apply a rule in generation mode, even though it can validate the correct answer. Goal projection's deterministic fallback applies the three remaining rules directly from their regex-parsed structure, bypassing the model entirely for those steps.

The unprovable case (B7): The three premises ("The meeting was held on a Tuesday", "All committee members received the agenda", "The conference room was booked in advance") are semantically isolated from the goal ("The committee will approve the proposal"). The main loop, all three sampling cycles, three backward chaining triggers, two goal projection triggers, and two pairwise search sweeps all find no valid chain. The system correctly stalls at `MAX_ITER` — crucially, with `false_proof_rate = 0.0`. The pairwise search (exhaustive over all premise pairs) reaching its cap without finding a goal-similar step is the strongest indicator that no valid derivation exists from the given store.

7. Conclusions and Future Directions

What the System Achieves

DRAS demonstrates that a **small fine-tuned LLM (7B parameters) combined with an iterative RAG loop and a multi-layer recovery stack can achieve 100% proof completion on a benchmark covering the full range of classical deductive inference patterns** — modus ponens, modus tollens, contrapositive chaining, disjunctive syllogism, and mixed combinations up to 6 steps — while maintaining zero false proof rate on unprovable problems.

The key architectural win is the separation of retrieval, inference, and validation. Each component can be improved independently: a better retriever improves premise selection without retraining the model; a stronger base model improves atomic step accuracy; the recovery mechanisms handle the structured failure modes of the current model without requiring those modes to be fixed in training.

Scalability Considerations

The architecture scales along two axes:

Step depth: The loop is depth-unlimited (only `max_iterations` is a hard stop). Increasing `max_iterations` allows longer proofs. The recovery mechanisms are triggered by consecutive rejects, not by proof depth, so they adapt to wherever the chain stalls.

Knowledge base size: The retriever is in-memory and dense. At larger scale, ChromaDB (used in the dense backend) supports persistent storage and approximate nearest-neighbor search at millions of documents. The $O(N^2)$ pairwise search is the only $O(N^2)$ operation and is triggered rarely; it could be replaced with a structured forward-chaining algorithm for production use.

Current Limitations

Modus ponens instantiation in generation mode: The model's most consistent failure is generating a conditional output when asked to apply a universal rule to a specific entity in generation mode. The deterministic fallback handles this reliably but is regex-dependent — it would fail on rules that do not match the `"Any X that/with [antecedent] [consequent]"` pattern. A more robust fix would be a targeted fine-tuning pass on modus ponens instantiation examples.

Open-world facts: All benchmarks are closed-world (all necessary premises are provided in seeds). Open-world reasoning — where the model would need to retrieve facts from external knowledge — requires a different retrieval setup.

Negation representation: The system handles negation in the premises (`"has not completed"`, `"is not cleared"`) but does so through learned pattern matching in the model rather than explicit symbolic negation. This works for the benchmark patterns but may not generalize to subtle negation interactions in more complex problems.

Architectural Forward Path

The two most impactful next steps are:

1. **Retrieve from a grounded external knowledge base** alongside the in-memory store — allowing the system to handle open-world problems where not all facts are given
2. **Train the model on modus ponens instantiation in generation mode** — targeted oracle examples showing `[entity_fact, universal_rule] → entity_conclusion` would close the gap that currently requires the deterministic fallback

The architecture's value proposition — auditable, attributed, step-by-step reasoning traces that a domain expert can inspect — is robust to both improvements. The proof trace format does not change; only the reliability and coverage of individual steps changes.

System: DRAS v1.0 — DeepSeek-R1-Distill-Qwen-7B-bnb-4bit + LoRA + dense RAG loop

Run command: `python scripts/run_eval.py --mode e2e --pair-search 12 --goal-proj 9`